Networking

OSI Model + TCP/IP Stack

↓

URL (Uniform Resource Locator) → web address that specifies how and where to access a resource on the internet.

~~type URL~~

URL components

http://example.com/product/electric/phone

- http → scheme
- example.com → domain
- /product/electric → path
- /phone → resource

- scheme → tells the browser to connect to the server using a protocol, in this ex it's http.
- domain name → name of the site
- path refers to complete leading path while resource is the item returned

[difference isn't fully accurate in terms of URL]

Flow

↓

User enter URL in the browser

↓

Browser needs to know how to reach the server (example.com), ~~or process~~ this is done with a process called DNS lookup which means finding the IP address that corresponds to a domain name

↓

to make the lookup process fast the DNS information is cached So Browser looks up IP in cache and if its not in it the browser asks the OS, OS also searches in its cache if not found OS makes query out to the internet to a DNS resolver (the server or system that does the process of DNS lookup)

↓

Resursive lookup

↓

DNS Server

untill the ip address is resolved

↓

Browser gets IP

↓

Browser establishes TCP connection with Server, handshake involved → keep alive connection used by modern browsers to reuse an established connection. If protocol HTTPS → process to establish connection more complicated & time consuming (SSL/TLS handshake expensive) → can use SSL session resumption to lower cost (time, cpu,)

↓

Once connection established browser sends HTTP request to server

↓

Server sends HTTP response after processing request

↓

Browser renders HTTP response or HTML content

↓

If additional resources like JS bundles & images browser repeats the entire process if resources are on different server domain other -wise it just continues from HTTP request

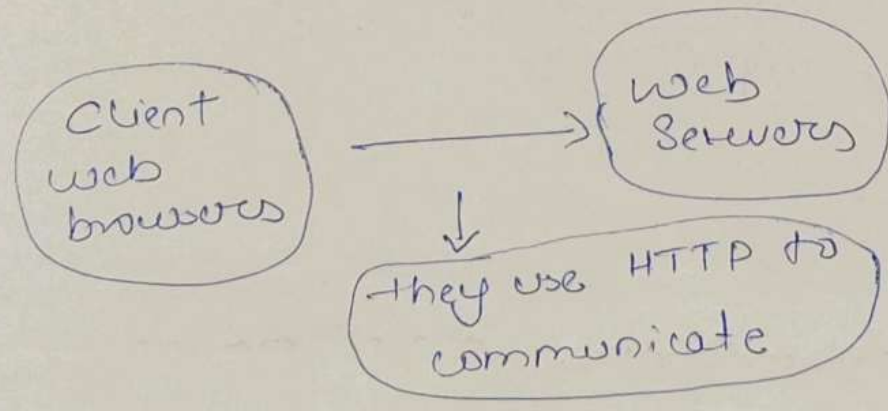
↓

Done

[Note: When OS sends DNS query to resolver and resolver returns IP, it returns the IP to OS and then the OS gives the IP to the browser, then Browser proceeds with TCP connection]

Application Layers

This layers provides network services directly to software applications that users interact with. These services can be HTTP/HTTPS, SMTP, FTP, SSH, etc



HTTP to be more specific is a set of rules that govern how two systems are supposed to talk to one another

(OR) → control or define

Its a set of rules that govern how a client & a web server communicate to request and transfer web resources

Basic working Flow

- Browser sends a HTTP request
- Server receives & processes & checks if it can fulfill the request
- Sends the response in sort of a yes or no (If file is there file is sent as a response otherwise an error represented in status codes)

Server always responds with a status code

Status codes :- three digit numbers that a web server sends back to a client to indicate the result of the request.

HTTP Status codes

-> 1xx -> Informational, basically information sent before the res final response
[Server gives this info to client's browser]

example:- server saying
"I have received your initial request
continue sending the remaining
part of the request"

ex:- huge file upload with request headers
↓
Server checks and responds with
1xx status code
↓
client sends remaining or the
actual data

-> 2xx -> Success

ex:- browser requests a page

GET /index.html

↓

Server responds

↓

HTTP/1.1 200 OK -> "I have found the page & I'm sending it to you"

-> 3xx -> Redirection, Server doesn't have files so it redirects to another server that has the files

example :-

Browser requests URL

↓

Server replies

```
HTTP/1.1 301 Moved Permanently
Location: https://newsite.com/about
```

↓

Basically means "The page has moved Go to this new address"

↓

Browser automatically requests the new URL

→ 4xx → Client error

most common → 401 (Page Not Found)

↓

"The request could not be completed because there is a problem with the client's request"

↓

Server looks for the requested file and if it doesn't find it sends this status code

→ 5xx → Server error

↓

Server failure while trying to process a valid request, basically problem on the server side

HTTP Request

Consists of:-

Request Line
Headers
Blank Line
Body

example:-

GET /index.html HTTP/1.1
host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html

Detailed info in order

↓

1) Request Line :- tells server what the client wants

GET /index.html HTTP/1.1
↓ method
↓ file (target)
↓ version of HTTP

"Using GET I want index.html, and I'm speaking HTTP/1.1"

2) Headers :- Name value pair info after request line
[Header-name: value]

ex:-

Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html

All these are considered as headers

[Header = key value pair]

The :- Host: www.example.com → tells the server which website do u want (one server can host many websites)

→ User-Agent :- identifies the client software but it can also contain OS info, rendering engine, compatibility tokens

example:-
Chrome
Firefox
Edge

- helps the servers customize responses

→ Accept :- Accept: text/html, basically means "i can accept HTML"

[The browser can list multiple formats it understands]

3) Blank line :- After all headers comes an empty line. Tells the server → "Headers are finished"

4) Body (optional) :- usualy empty in the request because its meant for response data, but can sometimes include data, ex:-

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/json

{
"username": "john",
"password": "1234"
}
```

] body sending login credentials

Note:- If there was no blank line servers would struggle to split the metadatas [HTTP parser used here]

↓

Part of the server that reads raw HTTP data & converts it into structured information the program can use

If the server can't differentiate between headers & additional info the structure is violated which results in request invalid, 400 Bad Request, ~~or~~ login failure

↓

to be more specific

↓

the parser does not think "structure is invalid"

Instead it does "I cannot determine where headers end so I cannot continue parsing safely"

So the real failure is not triggered by "structural violation"

but by loss of required delimeter (blank line) during streaming interpretation

↓

[short:- parsing failure due to missing delimeter]

Note:- HTTP request & response are not packets themselves

HTTP Response

Note:- HTTP request & response are not packets themselves

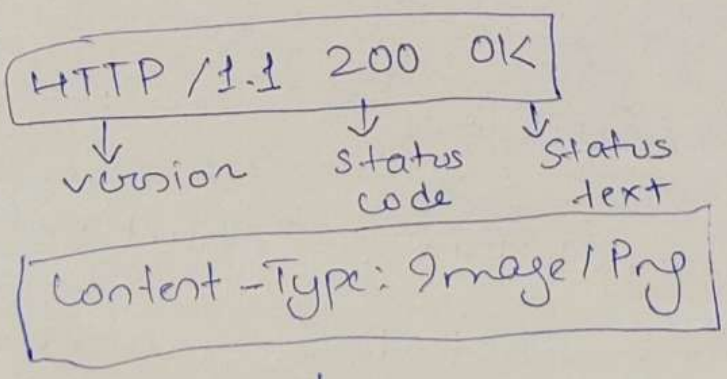
they are data carried inside packets, so they are protocol message

HTTP Response

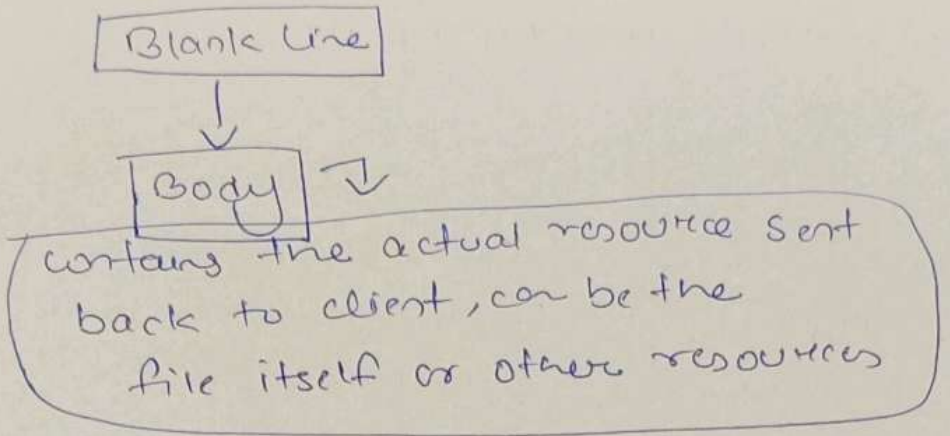
Components:-

- Status line or start line
- Headers
- Blank line
- Body

ex:-



↓



HTTP GET vs POST

GET :- method used to ~~retreive~~ retrieve data from the server without ~~mody~~ modifying it.

- data sent through URL
- mainly used for fetching info from servers
- can be bookmarked & stored in browser history
- not secure as visible in URL

POST :- method used to send data from the client (browser) to server for creating or updating resources, sent in ~~body~~ request body.

ex:- login data

"creating resources" ex:- • you sign up → new user created
• you upload pic → new file stored

"updating resources" ex:- • edit your profile name
• change password

Why Browsers use HTTP/HTTPS?

↓

because browsers need a standard way to communicate with servers over the internet and HTTP is the protocol designed for that job. Meanwhile HTTPS ensures security on top of HTTP